Arquitectura de eventos (schema `events`)

Objectivo

Um **diário append-only** de tudo o que importa rastrear entre domínios (OMS, MWM, segurança, `ref`, etc.), **sem depender de MongoDB**. O desenho da tabela `events.event` segue o mental de **um documento por evento** (como numa coleção Mongo ou num tópico Kafka), para facilitar **ETL futuro** para Mongo, Elastic ou outro sistema.

Postgres como fonte de verdade

Conceito Mongo / externo	Postgres `events.event`
`_id` ObjectId	`id` (BIGSERIAL) + `event_uuid` (UUID estável para exportação)
`insertedAt`	`recorded_at`
`occurredAt`	`occurred_at`
`type` / `name`	`event_type` (convenção `dominio.entidade.acção`)
`body` / `data`	`payload` (JSONB)
`metadata`	`metadata` (JSONB) — método HTTP, path, `correlation_id`, etc.

`aggregate_type` + `aggregate_id` permitem agrupar eventos por entidade de negócio (`operations.work_order`, `security.user`, …).

Schema e função canónica

- Script: `infra/sql/15_events_schema.sql`
- Tabela: `events.event`
- Função: `events.append_event(...)` — invocada **apenas a partir da aplicação** (sink em Node), não por triggers na base.

Aplicar no contentor `db`:

```
docker exec -i infra-db-1 psql -U tabgis_admin -d tabgis_db < infra/sql/15_events_schema.sql
```

Domínios canónicos

Lista em `backend/lib/events/constants.js` (`DOMAINS`): `oms`, `mwm`, `dms`, `ind`, `smt`, `tmp`, `aud`, `security`, `ref`, `gis`, `config`, `landbase`, `system`.

Novos domínios: acrescentar a `DOMAINS` e documentar o prefixo de `event_type` aqui.

Barramento plugável (Node)

- `backend/lib/events/createEventBus.js` — `createEventBus({ sinks })`, `createDefaultEventBus(pool)`.
- `backend/lib/events/createPgSink.js` — sink que chama `events.append_event`.
- **Desligar** persistência: `EVENTS_DISABLED=1` no ambiente da API (barramento vazio; HTTP continua igual).

Novos sinks (ex.: Mongo, fila)

1. Implementar `async function mongoSink(envelope) { ... }` com o mesmo contrato de `envelope` que `createPgSink`.
2. `createEventBus({ sinks: [createPgSink(pool), mongoSink] })`.
3. Garantir que erros são capturados **dentro** do sink (o barramento já isola falhas entre sinks).

API HTTP

Método	Rota	Descrição
GET	/api/events	Lista recente; query: limit, domain, event_type, aggregate_type, aggregate_id, since (ISO), correlation_id
GET	/api/events/export	**ETL / replicação:** mesmos filtros que /api/events, mais after_id (default 0) e limit (default 500, máx. 2000); ordenação **id ASC**; resposta { after_id, next_after_id, count, events }. Com **shape=mongo**, cada elemento de events segue o formato documentado em §Compatibilidade com Mongo (_id = event_uuid). Privilégio: module.aud (com SECURITY_ENFORCE=1, igual a /api/events).
PATCH	/api/schematic/views/:viewId/nodes/:nodeId	Actualiza só x e/ou y (par de coordenadas de foco do nó; emite evento com event_type **schematic.view_node.updated** e **domain** gis (requer privilégio module.esq quando SECURITY_ENFORCE=1)

Cabeçalho opcional em **todas** as mutações: X-Correlation-ID (UUID). Se omitido, a API gera um UUID por pedido e regista-o em metadata.correlation_id e na coluna correlation_id quando o sink persiste.

Eventos já emitidos (mutações)

event_type	Quando
security.user.created	POST /api/users
security.user.updated	PATCH /api/users/:id (ex.: activar/desativar colaborador)
security.user.group.updated	PATCH /api/users/:id/group
security.group.privileges.updated	PUT /api/security/groups/:id/privileges
oms.work_order.updated	PATCH /api/work-orders/:id
oms.work_order_task.created	POST /api/work-orders/:id/tasks
oms.work_order_task.updated	PATCH /api/work-orders/:id/tasks/:task_id
ref.types_batch.updated	PUT /api/ref/types
schematic.view_node.updated	PATCH /api/schematic/views/:viewId/nodes/:nodeId (domínio gis — vista esquemática / unifilar)

Outros domínios (DMS, IND, SMT, TMP, AUD, GIS, landbase) devem emitir **a partir de rotas REST** (ou de processos Node dedicados), com o mesmo contrato emitWithRequest / barramento.

Política: sem triggers PostgreSQL em events

Não se usam triggers AFTER INSERT/UPDATE para alimentar events.event. Motivos: manutenção mais simples, testes no mesmo processo que a API, evitar lógica duplicada e efeitos surpresa em cargas em massa ou psql.

Alternativas quando a alteração não passa por uma rota HTTP existente:

1. **Nova rota ou comando administrativo** na API que encapsule a operação e chame emitWithRequest no fim.

2. **Script ou job Node** (cron, worker) que use o mesmo `pool` + `createPgSink` / `eventBus.emit`, reutilizando `events.append_event`.
3. **Rever o fluxo** para que alterações relevantes passem sempre por camadas que já emitem eventos.

A função SQL `events.append_event` existe para ser chamada **a partir do código da aplicação** (sink), não como destino de triggers.

Compatibilidade com Mongo

Um job de exportação pode chamar em ciclo `GET /api/events/export?after_id=<cursor>&limit=500&shape=mongo` (ou ler directamente `SELECT … FROM events.event WHERE id > $cursor ORDER BY id ASC`). Usar `next_after_id` da resposta como próximo `after_id` até `count === 0`. Inserir na coleção `event_report` ou `events` com `_id` = `event_uuid` (dedupe idempotente).

Implementação de referência do mapeamento `shape=mongo`: `backend/lib/events/eventRowToMongoShape.js`.